

操作系统课程设计

基于 RISC-V xv6 的 操作系统机制实现与实验

MIT 6.S081 2020 全部 11 个 Labs

目标等级	A 级 - 完成全部实验内容
实验平台	Ubuntu 22.04/24.04 / RISC-V GCC 11.4/13.3 / QEMU 6.2/8.2
组号	1 组
成员	李胤龙（学号 2453195，单人组）
日期	2026 年 8 月

阅读导航

下列目录可点击跳转。Word 导航窗格和 PDF 书签可继续展开每个小课题的目的、步骤、问题与心得。

项目信息

摘要

1. 总体方案与环境搭建

- 1.1 需求分析
- 1.2 仓库组织
- 1.3 环境搭建
- 1.4 测试方法

2. Lab 1 : Unix Utilities

- 2.1 Boot xv6 (easy)
- 2.2 sleep (easy)
- 2.3 pingpong (easy)
- 2.4 primes (moderate/hard)
- 2.5 find (moderate)
- 2.6 xargs (moderate)

3. Lab 2 : System Calls

- 3.1 System call tracing (moderate)
- 3.2 Sysinfo (moderate)

4. Lab 3 : Page Tables

- 4.1 Print a page table (easy)
- 4.2 A kernel page table per process (hard)
- 4.3 Simplify copyin/copyinstr (hard)

5. Lab 4 : Traps

- 5.1 RISC-V assembly (easy)
- 5.2 Backtrace (moderate)
- 5.3 Alarm (hard)

6. Lab 5 : Lazy Page Allocation

- 6.1 Eliminate allocation from sbrk() (easy)
- 6.2 Lazy allocation (moderate)
- 6.3 Lazytests and Usertests (moderate)

7. Lab 6 : Copy-on-Write Fork

- 7.1 Implement copy-on-write fork (hard)

8. Lab 7 : Multithreading

- 8.1 Uthread: switching between threads (moderate)
- 8.2 Using threads (moderate)
- 8.3 Barrier (moderate)

9. Lab 8 : Locking

- 9.1 Memory allocator (moderate)
- 9.2 Buffer cache (hard)

10. Lab 9 : File System

- 10.1 Large files (moderate)
- 10.2 Symbolic links (moderate)

11. Lab 10 : mmap

11.1 mmap (hard)

12. Lab 11 : Network Driver

12.1 E1000 transmit (moderate)

12.2 E1000 receive (hard)

13. 自动化测试与实验结果

14. 项目难点总结

15. 总体实验心得

16. 参考资料

项目信息

项目	内容
课程	操作系统课程设计
选题	MIT 6.S081 2020 xv6 全部 Labs
目标等级	A 级（完成全部实验内容）
组号	1 组
姓名	李胤龙
学号	2453195
选课课号	4
成员与分工	单人组；独立完成环境搭建、代码实现、测试、文档与答辩材料
源码仓库	GitHub：xv6 全部 Labs 课程设计 (https://github.com/lilong555/xv6-all-labs-2020-course-project)

摘要

本项目以 MIT 6.S081 2020 的 RISC-V xv6 为基础，完成用户态工具、系统调用、页表、陷阱、惰性分配、写时复制、多线程、锁、文件系统、内存映射和网卡驱动共 11 个 Lab、28 个实验小题。项目持续修改内核的进程、内存、文件、同步与设备模块，并使用课程官方评分器逐项验收，最终取得 985/985 分。

源码按“一项 Lab 一个 Git 分支”组织，保留各实验规定的起点与评分脚本。文档分支 [course-project](#) 提供环境安装、全量评分和演示入口。实验主要在 WSL2 Ubuntu 22.04、RISC-V GCC 11.4、QEMU 6.2 上评分，并在 Ubuntu 24.04、GCC 13.3、QEMU 8.2 上完成兼容验证。

1. 总体方案与环境搭建

1.1 需求分析

课程说明将“完成全部实验内容”列为 A 级路线。本项目选择 2020 版 xv6 Labs，验收目标包括：全部实验有可追溯源码分支；能够在线编译和启动 xv6；官方评分全部通过；报告逐小题记录目的、步骤、问题与心得；答辩时可以现场演示典型内核机制。

1.2 仓库组织

course-project	文档、安装脚本、全量评分与答辩入口
riscv	RISC-V xv6 基线
util	Lab 1: Unix utilities
syscall	Lab 2: System calls
pgtbl	Lab 3: Page tables
traps	Lab 4: Traps
lazy	Lab 5: Lazy allocation
cow	Lab 6: Copy-on-write
thread	Lab 7: Multithreading
lock	Lab 8: Locking
fs	Lab 9: File system
mmap	Lab 10: mmap
net	Lab 11: Network driver

各 Lab 的框架并非来自同一个连续分支，强行合并会破坏官方评分基线。因此实验成果以独立分支交付，使用 `git diff reference/<lab>..<lab>` 查看该实验完整差异。

1.3 环境搭建

主要评分环境为 WSL2 Ubuntu 22.04（16 逻辑核、7.4 GiB 内存），兼容环境为 Ubuntu 24.04 服务器（2 核、3.8 GiB 内存）。安装依赖：

```
sudo apt-get update
sudo apt-get install -y qemu-system-misc gcc-riscv64-linux-gnu \
  binutils-riscv64-linux-gnu gdb-multiarch git make python3 python-is-python3
```

仓库提供等价脚本 `sudo ./scripts/setup-ubuntu.sh`。切换到某实验分支后执行 `make qemu` 启动，执行 `make grade` 评分。新版 GCC 会把 xv6 shell 的非返回递归诊断为 `-Winfinite-recursion`，项目通过编译器能力探测按需降级该警告；同时显式导出 `_entry` 入口，并在网络分支加入 QEMU 6 以上所需的 PMP 初始化。

1.4 测试方法

`scripts/grade-all.sh` 使用临时 Git worktree 隔离 11 个实验的构建目录，逐分支调用官方评分脚本并保存日志。现场演示可运行 `./scripts/demo-lab.sh cow` 等命令。完整评分结果见第 13 章。

2. Lab 1 : Unix Utilities

2.1 Boot xv6 (easy)

1) 实验目的

熟悉 RISC-V 交叉编译、QEMU 虚拟机、xv6 shell 以及课程评分工具的基本工作流。

2) 实验步骤

安装工具链后执行 `make qemu`，观察内核启动和 `init` 拉起 shell；在 xv6 中运行 `ls`、`cat README`，用 `Ctrl-a x` 退出；最后执行 `make grade` 验证环境。

3) 实验中遇到的问题和解决方法

GCC 11/13 对递归和入口符号的诊断比旧环境严格，导致 `-Werror` 构建失败。Makefile 先探测警告选项是否存在再添加兼容参数，并在 `kernel/entry.S` 中声明 `.globl _entry`。

4) 实验心得

先建立可重复的编译、运行、退出和评分闭环，能够把后续问题限定为代码问题，而不是环境问题。

2.2 sleep (easy)

1) 实验目的

练习用户程序参数解析和调用已有 `sleep` 系统调用。

2) 实验步骤

新增 `user/sleep.c`，检查参数数量，用 `atoi` 转换 `tick` 数并调用 `sleep`；将程序加入 Makefile 的 `UPROGS`，编译后分别测试正常参数和缺失参数。

3) 实验中遇到的问题和解决方法

xv6 的用户态库比标准 C 库精简，不能依赖完整的错误处理函数。程序通过明确的参数数量检查、向标准错误输出提示并返回非零状态处理非法调用。

4) 实验心得

即使是很小的命令，也需要把参数契约和退出状态处理完整；系统工具的可组合性依赖这些细节。

2.3 pingpong (easy)

1) 实验目的

理解 `pipe`、`fork`、`read`、`write` 和 `wait` 的协作关系。

2) 实验步骤

建立两个单向管道，父进程写入一个字节，子进程读取后打印并从另一管道回复；父进程 读取回复后等待子进程退出。

3) 实验中遇到的问题和解决方法

进程若保留自己不用的管道写端，读端可能永远等不到 EOF。`fork` 后立即关闭无关端点，并在通信完成后关闭剩余描述符。

4) 实验心得

管道不仅传递数据，也通过“所有写端均关闭”传递结束事件，因此文件描述符所有权是 进程通信设计的一部分。

2.4 primes (moderate/hard)

1) 实验目的

使用进程和管道实现并发素数筛，理解 Unix 管道的级联组合。

2) 实验步骤

父进程向首个管道写入 2 至 35；每级筛选器读取第一个数作为素数，创建子筛选器，只向 下一级转发不能被该素数整除的数；输入结束后逐级关闭管道并回收子进程。

3) 实验中遇到的问题和解决方法

递归筛选器容易因未关闭写端而阻塞，也可能因不等待子进程造成输出次序不稳定。解决方法是为每一级明确输入端、输出端和回收责任，写完立即关闭输出端，再 `wait`。

4) 实验心得

该实验把“一项任务一个进程”的思想具体化：简单进程通过管道连接后可以形成清晰的 流式并发结构。

2.5 find (moderate)

1) 实验目的

掌握 xv6 的目录项结构、文件状态查询和递归目录遍历。

2) 实验步骤

打开起始路径并读取 `struct dirent`；跳过空目录项、`.` 和 `..`；构造子路径后调用 `stat`，目录继续递归，普通文件比较末级名称并输出匹配路径。

3) 实验中遇到的问题和解决方法

目录名是固定 `DIRSIZ` 字节，不保证以零结尾，直接传给字符串函数会越界。复制名称时 预留终止字节并显式补零，同时在拼接前检查路径缓冲区长度。

4) 实验心得

文件系统接口背后仍是定长磁盘结构；用户程序必须理解数据表示，不能把磁盘字段直接 当作普通 C 字符串。

2.6 xargs (moderate)

1) 实验目的

理解标准输入、命令参数数组和 `fork/exec` 的组合。

2) 实验步骤

保留命令行中的基础参数，逐字符读取标准输入并按换行划分任务；为每行构造以空指针 结束的 `argv`，创建子进程执行目标命令，父进程逐项等待。

3) 实验中遇到的问题和解决方法

输入行、单词数量和参数数量均有上限，且 EOF 前最后一行可能没有换行。实现中检查缓冲区边界，在 EOF 时处理残留内容，并保证参数数组最后一项为零。

4) 实验心得

`xargs` 展示了 shell 工具通过统一的字节流和参数约定组合，不需要工具之间建立专用 接口。

3. Lab 2 : System Calls

3.1 System call tracing (moderate)

1) 实验目的

贯通用户包装函数、`ecall`、陷阱入口、系统调用分派和进程控制块。

2) 实验步骤

增加系统调用号和用户 stub；在 `proc` 中保存 trace 位掩码；`sys_trace` 设置掩码，`fork` 时复制；`syscall()` 执行目标调用后，按系统调用号检查位并打印 PID、名称和 返回值。

3) 实验中遇到的问题和解决方法

若在调用前打印就无法得到真实返回值，若 `fork` 不复制掩码则子进程失去跟踪状态。因此输出放在分派函数取得返回值之后，掩码纳入进程继承路径。

4) 实验心得

一个系统调用会跨越用户 ABI、汇编 stub、陷阱和内核分派多层边界；接口编号和参数 规则必须在每一层保持一致。

3.2 Sysinfo (moderate)

1) 实验目的

实现返回空闲内存和活动进程数的系统调用，练习内核状态统计与安全复制。

2) 实验步骤

在分配器锁保护下遍历空闲页链表计算字节数；遍历进程表统计非 `UNUSED` 项；先构造 内核态 `struct sysinfo`，再通过 `copyout` 写入用户地址。

3) 实验中遇到的问题和解决方法

共享链表可能并发变化，用户指针也可能无效。统计时持有对应锁，用户地址使用 `argaddr` 获取并统一交给 `copyout` 验证，禁止内核直接解引用。

4) 实验心得

系统调用既是功能接口也是保护边界；内核返回数据时同样必须防范不可信用户指针。

4. Lab 3 : Page Tables

4.1 Print a page table (easy)

1) 实验目的

理解 Sv39 三级页表的索引、PTE 标志和叶子/非叶子页表项区别。

2) 实验步骤

实现递归 `vmprint`：遍历每级 512 个 PTE，只输出有效项；无 R/W/X 权限的有效项视为 下一级页表，否则视为叶子映射；在指定进程的 `exec` 后打印页表。

3) 实验中遇到的问题和解决方法

仅凭 `PTE_V` 不能判断是否递归，否则会把物理页内容当页表解析。解决方法是同时检查 R/W/X 位，并按层级输出固定缩进以核对树结构。

4) 实验心得

页表不是平面数组，而是一棵按需分配的稀疏树；叶子权限位也是页表结构解释的一部分。

4.2 A kernel page table per process (hard)

1) 实验目的

为每个进程维护独立内核页表，理解 `satp`、TLB 与内核栈映射生命周期。

2) 实验步骤

进程创建时复制必要的设备和内核映射，并为其内核栈建立映射；调度进程时写入该页表的 `satp` 并执行 `sfence.vma`；回到调度器时恢复全局内核页表；进程释放时只释放页表页 和栈映射，不释放设备物理页。

3) 实验中遇到的问题和解决方法

错误地调用普通用户页表释放函数会释放设备或内核物理页。实现专用的页表页释放路径，并逐一核对 `allocproc`、失败回滚、`scheduler` 与 `freeproc` 的对称性。

4) 实验心得

页表切换不只是一条 CSR 指令，还涉及 TLB 失效和映射资源的完整生命周期。

4.3 Simplify copyin/copyinstr (hard)

1) 实验目的

把用户映射同步到进程内核页表，使内核可直接访问用户虚拟地址并简化复制路径。

2) 实验步骤

创建或增长用户地址空间后，把对应叶子 PTE 映射到进程内核页表并清除用户权限；收缩、释放、`fork` 和 `exec` 时同步维护；用直接复制版本替换逐页 `walkaddr` 的实现。

3) 实验中遇到的问题和解决方法

用户地址若增长到 PLIC 区域会与内核映射冲突，两套页表任一错误路径不同步也会留下悬空映射。实现统一边界检查，并在分配失败时按已完成范围逆序回滚。

4) 实验心得

优化复制操作扩大了需要维护的不变量：任何用户地址空间变化都必须同时更新两套页表。

5. Lab 4 : Traps

5.1 RISC-V assembly (easy)

1) 实验目的

理解 RISC-V 调用约定、栈帧、返回地址和调用者/被调用者保存寄存器。

2) 实验步骤

编译实验程序并阅读反汇编，定位函数参数所在的 `a0-a7`、返回地址 `ra`、栈指针 `sp` 和帧指针 `s0`；结合运行结果完成 `answers-traps.txt`。

3) 实验中遇到的问题和解决方法

优化后部分函数可能内联，源代码行与汇编不再一一对应。通过同时查看符号、地址和调用约定，关注实际控制流而不是机械匹配源代码行。

4) 实验心得

理解 ABI 后，陷阱保存现场、线程切换和 `backtrace` 中的寄存器布局就不再是孤立知识。

5.2 Backtrace (moderate)

1) 实验目的

沿内核栈帧输出调用链，为 `panic` 和内核调试提供定位信息。

2) 实验步骤

用内联汇编读取 `s0`；根据 ABI 从 `fp-8` 读取返回地址、从 `fp-16` 读取上一帧指针；以当前 4 KiB 内核栈页为边界迭代，并在 `panic` 中调用。

3) 实验中遇到的问题和解决方法

没有终止条件会越过内核栈访问非法地址。将初始帧指针对齐到页边界，以该页的上下界 校验每次迭代，并在帧指针不再单调前进时停止。

4) 实验心得

调试工具依赖编译器和 ABI 约定；掌握栈帧结构可以把原始地址转换为可追踪的调用历史。

5.3 Alarm (hard)

1) 实验目的

实现用户态周期 `alarm` 和 `sigreturn`，理解陷阱现场保存、恢复与防重入。

2) 实验步骤

在进程中保存周期、累计 tick、处理函数、处理中标志和 trapframe 备份；时钟中断到期 时备份完整现场并把 `epc` 改为 `handler`；`sigreturn` 恢复备份并清除处理中标志。

3) 实验中遇到的问题和解决方法

`handler` 运行时再次触发会覆盖唯一备份，返回值寄存器也可能被系统调用返回路径改写。通过防重入标志禁止嵌套，并让 `sigreturn` 返回恢复后的 `a0`，保证原程序状态不变。

4) 实验心得

异步事件的关键不是跳转到处理函数，而是完整保存和精确恢复被打断的执行上下文。

6. Lab 5 : Lazy Page Allocation

6.1 Eliminate allocation from sbrk() (easy)

1) 实验目的

将地址空间增长与物理页分配解耦，为按需分配建立基础。

2) 实验步骤

修改 `sys_sbrk`：正增长只检查溢出并增加 `p->sz`，不调用 `growproc` 分配页面；负增长 仍立即解除超出新边界的已有映射；启动 `echo hi` 观察首次缺页。

3) 实验中遇到的问题和解决方法

若负数也只修改大小，已映射物理页不会释放；若无符号加法处理不当会发生上下溢出。实现正负分支和严格边界验证，收缩路径继续调用解除映射逻辑。

4) 实验心得

进程声明的虚拟地址范围与实际占用的物理内存是两个独立状态。

6.2 Lazy allocation (moderate)

1) 实验目的

在用户首次访问合法但尚未映射的地址时分配物理页。

2) 实验步骤

在 `usertrap` 识别 `load/store page fault`，读取 `stval`，向下页对齐；校验地址属于进程 范围且不是栈保护区；申请清零页、建立用户读写映射，失败则释放临时页并终止进程。

3) 实验中遇到的问题和解决方法

最初实现把所有低于 `p->sz` 的地址都视为合法，可能映射空指针附近或 `guard page`。增加进程大小、栈边界、页表状态和溢出的联合检查，并集中到可复用的惰性分配函数。

4) 实验心得

缺页从异常变成正常控制流后，地址合法性判断必须比普通访问路径更严格。

6.3 Lazytests and Usertests (moderate)

1) 实验目的

使惰性分配兼容 `fork`、系统调用复制、地址空间收缩、越界访问和内存不足场景。

2) 实验步骤

让 `uvmcopy` 跳过合法空洞；让 `uvmunmap` 可忽略未映射页；在 `copyin`、`copyinstr` 和 `copyout` 跨页时按需补页；运行 `lazytests` 与完整 `usertests`。

3) 实验中遇到的问题和解决方法

系统调用在内核态访问用户缓冲区不会经过普通用户缺页路径，跨页复制因此失败。复制函数逐页检查映射，对合法惰性地址补页；OOM 和中途失败路径释放已分配资源并杀死进程。

4) 实验心得

引入惰性机制后，所有曾假设“范围内页面一定存在”的路径都必须重新审计。

7. Lab 6 : Copy-on-Write Fork

7.1 Implement copy-on-write fork (hard)

1) 实验目的

让父子进程在 `fork` 后共享物理页，只在写入时复制，降低时间和内存开销。

2) 实验步骤

`uvmcopy` 让父子映射同一物理页，把原可写页改为只读并设置软件 COW 位；分配器为每个物理页维护锁保护的引用计数；`store fault` 和 `copyout` 检测 COW，引用数大于 1 时复制，等于 1 时直接恢复写权限；更新 PTE 后刷新 TLB。

3) 实验中遇到的问题和解决方法

难点是引用计数、PTE 替换和错误回滚必须原子地保持一致。实现中缩短计数锁持有时间，先准备新页再替换 PTE；`uvmcopy` 中途失败时撤销已建立映射和引用，避免泄漏或重复释放。

4) 实验心得

COW 是跨模块不变量：页表权限、TLB、分配器计数、缺页处理和内核复制少一处都无法正确。

8. Lab 7 : Multithreading

8.1 Uthread: switching between threads (moderate)

1) 实验目的

实现用户级线程上下文切换，理解线程寄存器状态和独立栈。

2) 实验步骤

在线程结构保存 `ra`、`sp` 和 `s0-s11`；新线程分配独立栈并令初始 `ra` 指向线程函数；汇编 `thread_switch` 保存旧上下文、恢复新上下文，调度器在 `RUNNABLE` 线程间切换。

3) 实验中遇到的问题和解决方法

只保存临时寄存器或栈未按 ABI 对齐会产生随机错误。按照 RISC-V 被调用者保存寄存器 集合设计上下文，并保证栈顶对齐和线程返回路径有效。

4) 实验心得

线程切换的最小状态由 ABI 决定；内核调度和用户线程库共享相同的上下文切换原理。

8.2 Using threads (moderate)

1) 实验目的

修复 pthread 哈希表的并发丢失，并尽量保留并行性能。

2) 实验步骤

复现多线程 `put` 丢键；为每个 bucket 建立互斥锁，插入链表时只锁定目标桶；`get` 在插入完成后并行读取；比较单线程和多线程耗时及丢失键数量。

3) 实验中遇到的问题和解决方法

全局锁虽正确但把所有插入串行化。按 bucket 加锁使同桶链表互斥、不同桶并行，在零丢失的同时满足性能测试。

4) 实验心得

并发设计需要同时回答“保护什么数据”和“允许哪些操作并行”，正确性不是唯一指标。

8.3 Barrier (moderate)

1) 实验目的

使用 mutex 和 condition variable 实现可重复使用的线程屏障。

2) 实验步骤

线程进入屏障后增加到达计数；最后到达者把计数清零、推进轮次并广播；其他线程在 `while` 中等待轮次变化。运行多轮、多线程测试验证每轮均同步。

3) 实验中遇到的问题和解决方法

只检查计数会把相邻轮次混淆，使用 `if` 等待也不能抵御虚假唤醒。加入单调轮次编号，等待者循环检查自己进入时的轮次是否已改变。

4) 实验心得

条件变量表示“状态可能变化”，真正的正确条件必须由共享谓词在锁内循环确认。

9. Lab 8 : Locking

9.1 Memory allocator (moderate)

1) 实验目的

降低多核分配和释放物理页时对单一 `kmem` 锁的竞争。

2) 实验步骤

把空闲页拆为每 CPU 一条链表并分别加锁；释放时放入当前 CPU；分配优先取本地页，本地为空时从其他 CPU 偷取一批；运行 `kallocetest` 和 `usertests`。

3) 实验中遇到的问题和解决方法

读取 `cpuid()` 后若线程迁移，会操作错误链表。调用 `push_off` 禁止中断，直到完成本地编号读取和链表操作；偷取时一次只按确定顺序持有必要锁，避免循环等待。

4) 实验心得

分区能把常见路径变成局部操作，但必须设计低频的跨区再平衡和明确的 CPU 归属规则。

9.2 Buffer cache (hard)

1) 实验目的

降低文件系统块缓存全局锁竞争，同时保持每个磁盘块最多一个缓存副本。

2) 实验步骤

按 $(dev, blockno)$ 哈希为多个 bucket，每桶独立加锁；命中时增加引用；未命中时选择 `refcnt==0` 的最久未使用块并迁移到目标桶；运行 `bcachetest`、大文件和用户测试。

3) 实验中遇到的问题和解决方法

跨桶迁移可能死锁，释放锁后再插入又可能生成重复块。使用额外淘汰锁串行化稀有的全局选择过程，桶内常规命中仍并行，并在确定受害块后再次核对目标块状态。

4) 实验心得

高性能锁设计通常优化高频命中路径，把复杂但低频的协调集中到可证明正确的慢路径。

10. Lab 9 : File System

10.1 Large files (moderate)

1) 实验目的

增加双重间接块，使 inode 能支持远大于原上限的文件。

2) 实验步骤

保留 11 个直接块、1 个一级间接块，并把一个槽作为二级间接块；扩展 `bmap` 按逻辑块号逐级索引并按需分配；扩展 `itrunc` 释放数据块、一级索引块和二级索引块。

3) 实验中遇到的问题和解决方法

二级索引的商余计算和缓冲区释放容易出错。按“直接、一级、二级”分段扣减块号，每次 `bread` 后确保所有成功与错误路径都 `brelease`，截断时逐层遍历并清零指针。

4) 实验心得

多级索引用少量 inode 空间换取大寻址能力，但分配与回收必须保持完全对称。

10.2 Symbolic links (moderate)

1) 实验目的

实现 `symlink(target, path)` 及 `open` 的符号链接解析。

2) 实验步骤

增加系统调用、`T_SYMLINK` 类型和 `O_NOFOLLOW`；创建链接 inode 并把目标路径写入其数据；`open` 默认循环读取目标并重新查找，指定 `O_NOFOLLOW` 时返回链接本身。

3) 实验中遇到的问题和解决方法

自环或多节点链接环会无限解析。限制最大跟随深度，每次切换 inode 时正确解锁并释放旧引用；所有目录和 inode 修改均放在日志事务中。

4) 实验心得

符号链接保存的是名字而不是 inode 引用，因此解析时必须重新经过路径查找和权限边界。

11. Lab 10 : mmap

11.1 mmap (hard)

1) 实验目的

实现文件支持的惰性内存映射、解除映射、共享写回和进程生命周期处理。

2) 实验步骤

进程维护固定数量 VMA，记录地址、长度、保护位、标志、文件偏移和引用；`mmap` 只登记元数据，访问时由 page fault 分配清零页并从文件读取；`munmap` 支持头部、尾部和整体解除，`MAP_SHARED` 可写页分段写回；`fork` 复制 VMA 并增加文件引用，`exit` 全部解除。

3) 实验中遇到的问题和解决方法

未 fault-in 的页不能当作错误，单次写回过大又会超过 xv6 日志容量。解除映射逐页检查 PTE，只处理实际映射页；写回按日志允许的大小分段执行，且 VMA 拆分只接受实验规定范围。

4) 实验心得

`mmap` 把文件、虚拟内存、缺页、日志和进程复制连接在一起，是资源引用和惰性机制的综合实验。

12. Lab 11 : Network Driver

12.1 E1000 transmit (moderate)

1) 实验目的

完成 E1000 发送描述符环，理解软件向设备移交缓冲区所有权的过程。

2) 实验步骤

读取 TDT 指向的描述符，检查 DD 确认旧发送完成；释放旧 mbuf，写入新 mbuf 的物理地址和长度，设置 EOP|RS，最后推进 TDT 通知网卡。

3) 实验中遇到的问题和解决方法

描述符忙时覆盖会造成硬件读取已释放内存。发送路径在持锁状态检查 DD，忙则返回失败；只有完成描述符的旧 mbuf 可以释放，寄存器更新放在描述符写完之后。

4) 实验心得

设备驱动的顺序来自所有权协议：先准备内存描述符，再通过寄存器把工作正式交给硬件。

12.2 E1000 receive (hard)

1) 实验目的

完成接收环处理，把收到的以太网帧交给网络栈并持续补充缓冲区。

2) 实验步骤

从 RDT 后一个描述符开始，循环处理带 DD 的项；记录长度并把 mbuf 交给 net_rx；为描述符分配替换 mbuf、清状态，更新 RDT 归还硬件，以下标取模继续。

3) 实验中遇到的问题和解决方法

上交网络栈后原 mbuf 已不属于接收环，继续复用会产生双重所有权。每处理一项立即安装新 mbuf，再更新尾指针；同时加入官方 PMP 初始化修复，解决 QEMU 6+ 无法进入 S-mode。

4) 实验心得

环形队列的下标并不难，真正关键的是在 CPU、网卡和网络栈之间准确转移缓冲区所有权。

13. 自动化测试与实验结果

全部 Lab 使用各自分支携带的官方 `grade-lab-*` 评分器测试。主要环境完整结果如下：

实验分支	官方得分	结果
util	100/100	通过
syscall	35/35	通过
pgtbl	66/66	通过
traps	85/85	通过
lazy	119/119	通过
cow	110/110	通过
thread	60/60	通过
lock	70/70	通过
fs	100/100	通过
mmap	140/140	通过
net	100/100	通过
合计	985/985	全部通过

执行全量评分：

```
./scripts/grade-all.sh
```

评分器覆盖正常输出、非法参数、完整 `usertests`、内存不足、并发压力、锁竞争统计、文件系统大文件与网卡收发。服务器为 2 核，部分固定时限性能项会受硬件影响，因此正式分数采用本机 16 逻辑核环境的完整结果，服务器用于编译、QEMU 启动和兼容验证。

14. 项目难点总结

1. **跨模块不变量**：COW 同时涉及 PTE 权限、TLB、物理页引用、缺页和 `copyout`。
2. **资源生命周期**：页表页、文件、inode、buffer、mbuf 与锁的成功/失败路径必须对称。
3. **并发正确性和性能**：锁实验不仅要求无竞争错误，还要求降低 test-and-set 次数。
4. **惰性机制**：lazy allocation 和 mmap 要求所有访问用户内存的路径容忍合法空洞。
5. **工具链演进**：需要区分真实内核缺陷与新版 GCC/binutils/QEMU 的兼容问题。

15. 总体实验心得

完成全部实验后，我对操作系统的理解从“模块功能”转变为“状态与不变量”。系统调用 看似只是增加一个编号，实际上贯穿用户 stub、陷阱、参数复制、权限和进程生命周期；一次页错误也不只是分配内存，而是页表、TLB、物理页所有权和错误恢复的联合操作。

xv6 代码量不大，可以沿一次调用追踪到硬件边界。COW、buffer cache 和网卡 ring 又说明，即使在教学内核中，并发和资源所有权仍是最容易发生严重错误的部分。官方评分器 将正常路径、边界、压力和性能约束统一纳入验收，使每项实现不仅“能运行”，而且能在 异常与并发条件下保持设计不变量。

16. 参考资料

1. MIT 6.S081 Operating System Engineering, Fall 2020 :
<https://pdos.csail.mit.edu/6.828/2020/xv6.html>
2. Russ Cox, Frans Kaashoek, Robert Morris, xv6: a simple, Unix-like teaching operating system。
3. RISC-V International, The RISC-V Instruction Set Manual。
4. Intel, 8254x Family of Gigabit Ethernet Controllers Software Developer's Manual。
5. 《24 级操作系统课程设计课程说明-王老师嘉定班》。
6. 《xv6 及 Labs 课程项目（2026）》。